

```
Notes:
• HW due Nov 6

Last class:
• Plotting with matplotlib
• Use pdf or other vector graphics formats when quality matters
• customizing matplotlib plots using rcParams and kwargs
• dictionaries

In [5]: import matplotlib.pyplot as plt
import matplotlib as mpl
my_plot_rc = {'font.size': 18,
              'lines.linewidth': 2,
              'savefig.dpi': 'tight'}
mpl.rcParams.update(my_plot_rc)

Final conclusion from last class:
• sometimes tables are more effective than graphs, particularly when we have small data sets and want to make fine-grained comparisons

Other times, looking at raw data is not so informative. Look at anscombe.py, which defines 4 data sets, known as "Anscombe's Quartet"

Looking at data, it seems to be just a mishmash of numbers. Can load the data and look at some basic statistics:
```

```
In [1]: import anscombe
df1, df2, df3, df4 = anscombe.quartet()
df1.describe()

Out [1]:
```

	x	y
count	11.000000	11.000000
mean	9.000000	7.500909
std	3.316625	2.031568
min	4.000000	4.260000
25%	6.500000	6.315000
50%	9.000000	7.580000
75%	11.500000	8.570000
max	14.000000	10.840000

```
df2.describe()

Out [2]:
```

	x	y
count	11.000000	11.000000
mean	9.000000	7.500909
std	3.316625	2.031657
min	4.000000	3.100000
25%	6.500000	6.695000
50%	9.000000	8.140000
75%	11.500000	8.950000
max	14.000000	9.260000

```
df3.describe()

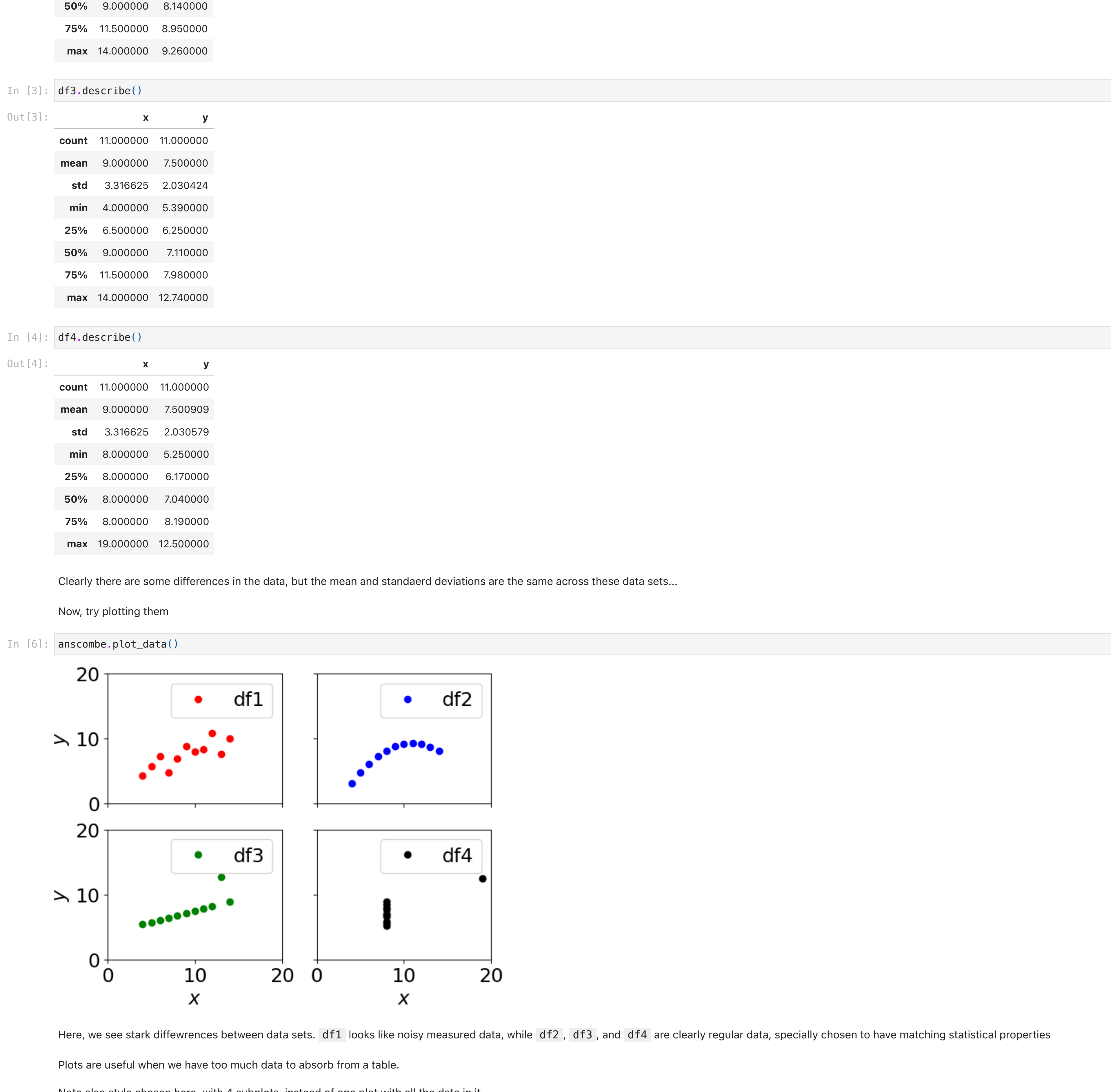
Out [3]:
```

	x	y
count	11.000000	11.000000
mean	9.000000	7.500000
std	3.316625	2.030424
min	4.000000	5.390000
25%	6.500000	6.250000
50%	9.000000	7.110000
75%	11.500000	7.980000
max	14.000000	12.740000

```
df4.describe()

Out [4]:
```

	x	y
count	11.000000	11.000000
mean	9.000000	7.500909
std	3.316625	2.030579
min	8.000000	5.250000
25%	8.000000	6.170000
50%	8.000000	7.040000
75%	8.000000	8.190000
max	19.000000	12.500000

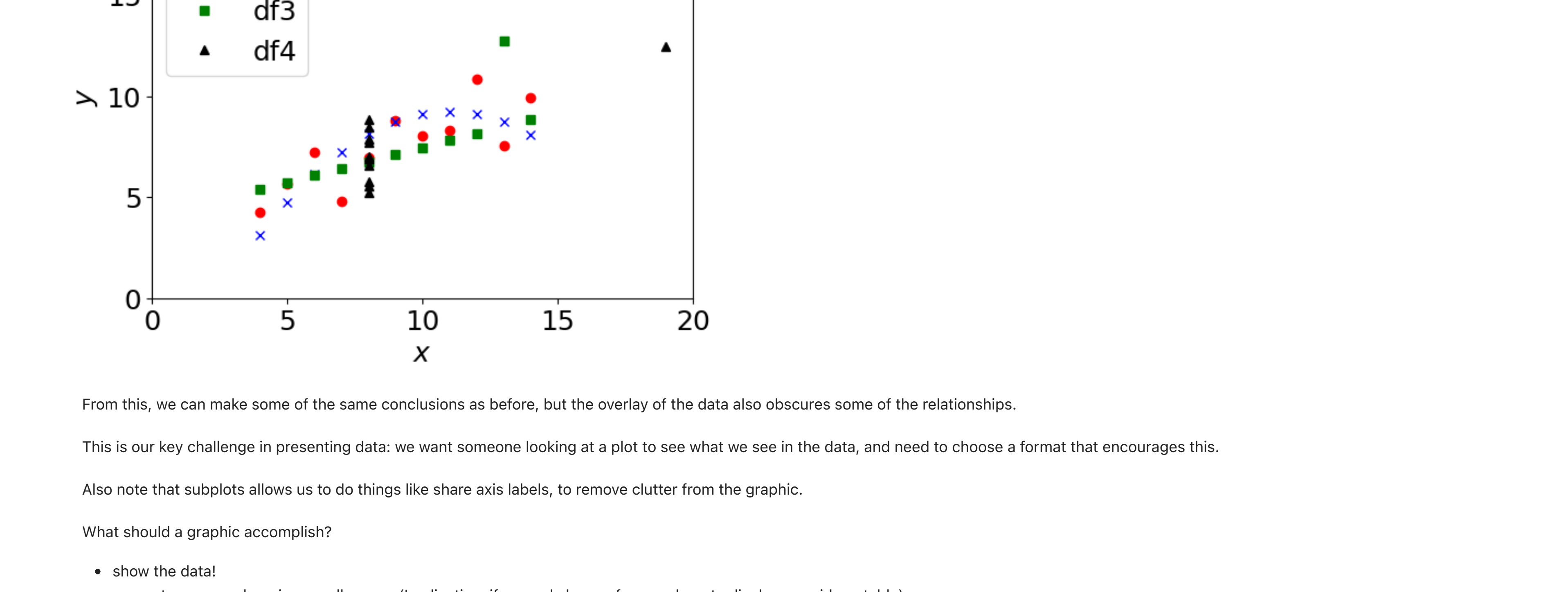


Here, we see stark differences between data sets. `df1` looks like noisy measured data, while `df2`, `df3`, and `df4` are clearly regular data, specially chosen to have matching statistical properties

Plots are useful when we have too much data to absorb from a table.

Note also style chosen here, with 4 subplots, instead of one plot with all the data in it.

Contrast this with:



From this, we can make some of the same conclusions as before, but the overlay of the data also obscures some of the relationships.

This is our key challenge in presenting data: we want someone looking at a plot to see what we see in the data, and need to choose a format that encourages this.

Also note that subplots allows us to do things like share axis labels, to remove clutter from the graphic.

What should a graphic accomplish?

- show the data!
- present many numbers in a small space. (Implication: if you only have a few numbers to display, consider a table)
- make large data sets coherent
- avoid distorting what the data naturally have to say. (Think about bad news graphics: misleading scalings, normalization. Data is misrepresented to fit a narrative)

- A graph should serve a reasonably clear purpose. Does it tabulate data? Describe or explore them? Is it decorative?

- Induce the viewer/reader to think about the substance of the data (rather than the design/production of the graphic)

- encourage us to compare different pieces of data
- reveal the data at several levels of detail

- be closely integrated with a description of the data. Never just show a graphic. In a report/manuscript, each graphic must have a clear caption describing what is being shown in the graph (explaining anything that's not clear from the legend). A caption is not just a title for the graph! Further, the text of the manuscript should include a description of the graphic and what the important conclusions are that the you want the reader to draw from the data.

Any time you include a graph in a report or presentation, ask yourself what is the purpose of the graphic?

- Is it to directly communicate/tabulate data? (I.e., you measured/produced a bunch of data and are representing it graphically?)
- Is it there to aid analysis or narration? (What "story" are you hoping to tell with the graphic?)

What goes into a good plot?

- Data
 - Need something to plot!
 - Gather and select (curate) data to actually plot
- Variable transformations
 - What makes sense for pre-processing the data before we plot it?
 - Should we sort the data? Does it already have a natural ordering (e.g., by time) or is there a way to sort it (e.g., by frequency or magnitude) that reveals information?
 - Should we normalize the data? (E.g., counts relative to population size, instead of raw counts?)
 - Should we average or aggregate variables to account for statistical noise?
- Is the difference from a best-fit curve (e.g., linear regression) more meaningful than raw data?
- With financial and similar data, how do we account for effects of inflation or other changes over time?

- Data frame:
 - On what space does the data belong?
 - 2D scatter plot / line graph?
 - 3D (surface) plot?
 - data on regional or world map?
- Data scale:
 - Is data linear in scale or do we need (another) transformation to visualize it effectively?
 - log-scale plots are better when we have data that varies over several orders of magnitude. Consider log-scaling of one or both axes. Use "symlog" scaling if data has positive and negative signs, but otherwise wide range
 - "logit" scale if data is between 0 and 1, but approaches close to both limits
 - Can define custom scales if needed, but should be careful not to distort data, or make data difficult for reader to interpret.
- Coordinates
 - Standard graphs are often in Cartesian (x-y) coordinates, with an independent variable along the x-axis and dependent variable(s) along the y axis, suggesting relationship $y = f(x)$.
 - Sometimes, illustrative to change this, or to change direction for "positive" x or y coordinates
 - Consider aspect ratio: Standard plots are often in a 4:3 ratio (not widescreen), but some data makes more sense in "taller" or "wider" graphs. Some data has geometric shape built into it (e.g., pixels in an image)
 - Also consider whether plotting x vs $f(x)$ or $f(x)$ vs $f(x)$ is more meaningful. What relationships do we expect to see in our data?
 - Sometimes consider polar or other plot coordinates
 - For some data, bar graphs or histograms are more appropriate (e.g., when data has discrete groupings, a connected line graph can be misleading). Then the x-coordinate can represent discrete groups
- Graph:
 - Actual data representation itself, as points and/or lines, markers, etc.
 - Generally choose these to try and convey information, with structured choice of these elements to enable consistent comparisons across graphics.
- Guides:
 - aids to interpreting the data
 - axes, including markers and ticks
 - grid lines, if helpful
 - labels and legends
 - indications of origin and compass directions from it
 - scaling lines (what pattern is expected from the data?)

Always be conscious of "chart junk". All elements in a plot should serve a purpose. Just because you can add a feature, doesn't mean that you should...

Also be conscious of viewers with reduced color vision. Red-green colorblindness is common (particularly in men), so assume it affects at least some of your viewers.

- use line styles and markers in addition to colour-coding data
- use thicker line widths
- consider contrasts in brightness, not just colour
- don't rely on shades to convey information.

Two useful strategies:

1. Print a figure in greyscale (on a black and white printer) to see if you can still distinguish data
2. Use a colour blindness simulator to see how an image looks to someone with reduced colour vision

In practice, develop a set of good habits, then do not need to do this with every figure you make.

Let's look at some plots and think about their construction.

```
In [8]: from IPython import plt
display.Image("bar_graph.png")

Out [8]:
```



This data is comparing two algorithmic variants (using a tie-breaker or not) on number of failures in an algorithm.

Data ranges from 0 (no failures) to a few hundred.

How do we plot this? Look at `bargraph.py`

- define data
- import matplotlib and set aspect ratio. Note: `figure, figsize` sets width and height in inches. Try to use the size that you want to use for your report, to avoid rescaling things later on.
- now plot with 2 subfigures, share y axis, plot bar graphs, use `axis, text` to put text on your graph by (x,y) location, text string, vertical and horizontal alignments
- subplot and axis labels
- log scaling on both y-axes
- set limits
- set "ticks" to label axes

Important note: matplotlib has different commands for setting properties on the axes of a subplot than directly on a figure. Be aware of this!

Closely related to bar graphs are histograms, which show frequencies of values to allow us to assess probabilities.

Simple example:

- use numpy module to generate normally distributed random numbers. (More on numpy later in the class, just use some functions from it for now for convenience.)

```
In [9]: import numpy as np

In [10]: data = np.random.normal(size=1000)
plt.hist(data)

Out [10]: (array([ 2., 17., 52., 139., 249., 237., 179., 92., 25., 8.]),
array([-3.43968969, -2.78514894, -2.13068819, -1.47622743, -0.82176668,
-0.16738592, 0.48715483, 1.14161358, 1.79687634, 2.45853789,
3.18499785]),
<BarContainer object of 10 artists>)
```



By default, divides data into 10 bins, which is quite coarse

```
In [11]: plt.hist(data, bins=50)

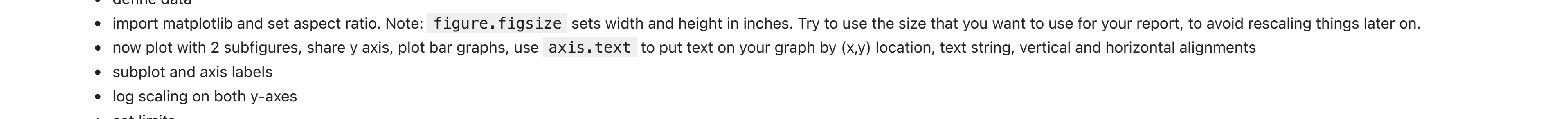
Out [11]: (array([ 1., 1., 0., 0., 0., 1., 2., 1., 0., 5., 5., 5., 14.,
11., 17., 16., 23., 27., 31., 42., 46., 30., 58., 57., 58., 46.,
63., 36., 50., 42., 38., 32., 45., 39., 25., 30., 22., 14., 14.,
12., 7., 5., 7., 3., 3., 2., 1., 2., 1., 2.]),
array([-3.43968969, -3.38871754, -3.17782539, -3.04693324, -2.91684189,
-2.78514894, -2.65425679, -2.52336464, -2.39247249, -2.26158834,
-2.13068819, -1.99979683, -1.86898388, -1.73881173, -1.60711958,
-1.47622743, -1.34533528, -1.21444313, -1.08355098, -0.95265883,
-0.82176668, -0.69087453, -0.55998238, -0.42989023, -0.29819887,
-0.16738592, -0.03641377, 0.09447838, 0.22537853, 0.35626268,
0.48715483, 0.61804698, 0.74893913, 0.87983128, 1.01072343,
1.14161358, 1.27258774, 1.40339989, 1.53429284, 1.66518419,
1.79687634, 1.92698849, 2.05788864, 2.18875279, 2.31964494,
2.45853789, 2.58142924, 2.71232139, 2.84321355, 2.9741857 ,
3.10499785]),
<BarContainer object of 50 artists>)
```



By default bins are touching, can add spacing by controlling relative width

```
In [12]: plt.hist(data, bins=50, rwidth=0.8)

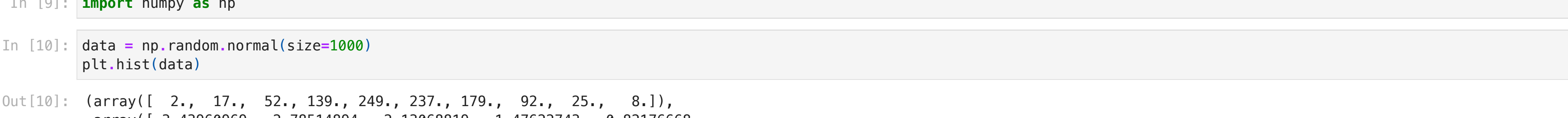
Out [12]: (array([ 1., 1., 0., 0., 0., 1., 2., 1., 0., 5., 5., 5., 14.,
11., 17., 16., 23., 27., 31., 42., 46., 30., 58., 57., 58., 46.,
63., 36., 50., 42., 38., 32., 45., 39., 25., 30., 22., 14., 14.,
12., 7., 5., 7., 3., 3., 2., 1., 2., 1., 2.]),
array([-3.43968969, -3.38871754, -3.17782539, -3.04693324, -2.91684189,
-2.78514894, -2.65425679, -2.52336464, -2.39247249, -2.26158834,
-2.13068819, -1.99979683, -1.86898388, -1.73881173, -1.60711958,
-1.47622743, -1.34533528, -1.21444313, -1.08355098, -0.95265883,
-0.82176668, -0.69087453, -0.55998238, -0.42989023, -0.29819887,
-0.16738592, -0.03641377, 0.09447838, 0.22537853, 0.35626268,
0.48715483, 0.61804698, 0.74893913, 0.87983128, 1.01072343,
1.14161358, 1.27258774, 1.40339989, 1.53429284, 1.66518419,
1.79687634, 1.92698849, 2.05788864, 2.18875279, 2.31964494,
2.45853789, 2.58142924, 2.71232139, 2.84321355, 2.9741857 ,
3.10499785]),
<BarContainer object of 50 artists>)
```



Can also do cumulative histogram:

```
In [13]: plt.hist(data, bins=50, rwidth=0.8, cumulative=True)

Out [13]: (array([ 1., 2., 2., 2., 2., 2., 3., 5., 5., 14.,
19., 24., 29., 43., 54., 71., 87., 110., 137.,
168., 210., 256., 286., 344., 401., 459., 505., 568.,
684., 634., 695., 734., 765., 811., 858., 875., 965.,
927., 941., 935., 907., 974., 979., 986., 989., 992.,
994., 995., 997., 998., 1000.]),
array([-3.43968969, -3.38871754, -3.17782539, -3.04693324, -2.91684189,
-2.78514894, -2.65425679, -2.52336464, -2.39247249, -2.26158834,
-2.13068819, -1.99979683, -1.86898388, -1.73881173, -1.60711958,
-1.47622743, -1.34533528, -1.21444313, -1.08355098, -0.95265883,
-0.82176668, -0.69087453, -0.55998238, -0.42989023, -0.29819887,
-0.16738592, -0.03641377, 0.09447838, 0.22537853, 0.35626268,
0.48715483, 0.61804698, 0.74893913, 0.87983128, 1.01072343,
1.14161358, 1.27258774, 1.40339989, 1.53429284, 1.66518419,
1.79687634, 1.92698849, 2.05788864, 2.18875279, 2.31964494,
2.45853789, 2.58142924, 2.71232139, 2.84321355, 2.9741857 ,
3.10499785]),
<BarContainer object of 50 artists>)
```



Many more options to control bin definitions, density vs. count, weights on data, etc.

More options are provided by seaborn library that has some specialized functionality for statistical data visualization.

```
In [15]: import seaborn as sns

In [16]: sns.histplot(data, bins=50)

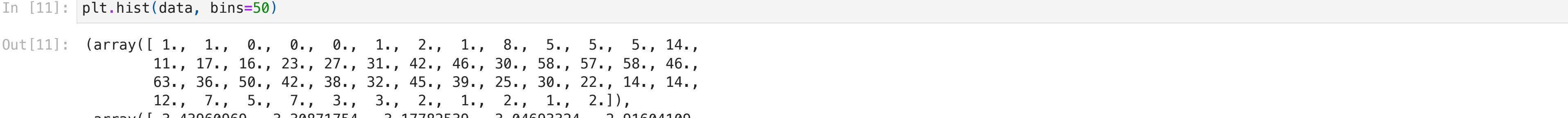
Out [16]: <Axes: ylabel='Count'>
```



For example, this allows us to plot a "kernel density estimate", a statistical estimate of the probability density function that generated the data.

```
In [17]: sns.histplot(data, bins=50, kde=True)

Out [17]: <Axes: ylabel='Count'>
```



In [18]: display.Image("histogram.png")

Out [18]:

